

Nome:

Número:

- Este teste contém 7 questões em 4 páginas e tem a duração de 2h.
- Responda às questões no espaço respectivo, indicando sempre um tipo para a função definida.
- Se nada for dito em contrário, pode usar funções auxiliares e/ou do prelúdio-padrão.

1. (3 valores) Para cada uma das seguintes perguntas, indique a resposta correta. *Respostas erradas não serão penalizadas.*

(a) Qual das seguintes afirmações é falsa?

- A. "Ola Mundo!!!" :: [Char] ☐
- B. "True" :: Bool ☐
- C. ['1','2','3'] :: String ☐

(b) Qual o comprimento da lista produzida pela expressão [1,2]:[3]:[4,5]:[]?

- A. 3 ☐
- B. 4 ☐
- C. 5 ☐

(c) Qual o resultado da expressão zip [1,2,3] "abracadabra"?

- A. [(1,'b'),(2,'r'),(3,'a')] ☐
- B. [(1,"abra"),(2,"cada"),(3,"bra")] ☐
- C. [(1,'a'),(2,'b'),(3,'r')] ☐

(d) Qual das seguintes definições utilizando listas em compreensão é equivalente à seguinte definição recursiva?

f [] = 0
f (x:xs) = 2*x + f xs

- A. f xs = sum [2*x | x<-xs] ☐
- B. f xs = product [2*x | xs<-xs] ☐
- C. f (x:xs) = 2*x + sum [x | x<-xs] ☐

(e) Indique um tipo admissível para a função definida por f n xs = drop n (reverse xs):

- A. Int -> ([Char], [Char]) ☐
- B. Int -> [Char] -> Char ☐
- C. Int -> [Char] -> [Char] ☐

(f) Considere a seguinte função em Haskell:

???

g x y | x <= y = g x (y-1)
 | otherwise = x + y

Qual das seguintes declarações de tipo, não pode ser usada em ????

- A. g :: Float -> Float -> Float ☐
- B. g :: a -> a -> a ☐
- C. g :: Int -> Int -> Int ☐

2. (3 valores) Responda a cada uma das seguintes questões, indicando **apenas** o resultado de cada expressão.

(a) Indique o resultado de `[1,3,4,5,6,9] !! 3`

R: _____

(b) Indique em extensão a lista `[(x,y) | x <- [1..3], y <- [2..4], (x+y) == 5]`.

R: _____

(c) Indique em extensão a lista `[1 | x <- [1..3], y <- [1..x]]`

R: _____

(d) Dada a seguinte função:

```
f 1 = reverse (f' 1)
  where f' [] = []
        f' (x:xs) = (2*x) : (f' xs)
```

Indique o valor de `f [1,2,3,4]`

R: _____

(e) Indique um tipo admissível para a expressão `[(x,x!!n) | n <- [0..10]]`:

R: _____

(f) Considere a seguinte definição em Haskell

```
f 0 = 0
f n = n*f (n-1)
A avaliação da expressão f 5 tem como resultado
```

R: _____

3. (4 valores) Escreva uma função `ordem` que dada um triplo de inteiros, `(i,j,k)`, retorna a ordem pela qual `i`, `j` e `k` estão ordenados da esquerda para a direita. Os resultados possíveis são "Crescente", "Todos Iguais", "Decrescente" ou "Não ordenados".

Nome:

Número:

4. (3 valores) Escreva uma função **duplica**, que dada uma lista **xs** devolve uma lista que é duas vezes maior do que **xs** e em que cada elemento da lista original aparece duas vezes seguidas na lista resultado. Por exemplo **duplica [2,3,4] = [2,2,3,3,4,4]** e **duplica "abc" = "aabbcc"**.

5. (3 valores) Considere uma função **quadradosImp** que dada uma lista de inteiros devolve uma lista de inteiros que substitui cada ímpar da lista original pelo quadrado desse elemento. Os elementos pares não são alterados. Por exemplo **quadradosImp [1,2,4,3,5] = [1,2,4,9,25]**.
- a) Defina **quadradosImp** recursivamente.
 - b) Defina **quadradosImp** usando listas em compreensão.

6. (2 valores) Defina a função `mesmaPos`, que dadas duas listas `xs` e `ys` retorna uma lista dos elementos que aparecem na mesma posição nas duas listas. Por exemplo, `mesmaPos [1,2,3,4,5] [5,4,3,2,1,] = [3]` e `mesmaPos "banana" "batata" = "baaa"`.

7. (2 valores) O problema de decompor uma quantia em trocos pode ser formalizado da seguinte maneira: dado um natural `n` e uma lista de naturais `xs`, encontrar decomposições de `n` como soma de valores em `xs` (eventualmente com repetições). Por exemplo, para `n=25` e `xs=[2,5,10]` uma decomposição possível é `[5,10,10]` (porque $25 = 5+10+10$). Escreva uma definição duma função `decompor` tal que `decompor n xs` encontra todas as alternativas para o problema dos trocos. O resultado deverá ser a lista vazia quando o problema não tem solução.